

[← Back to Course Schedule](#)

Final Project: Health and Fitness Club Management System

EGEN5208W Databases for Soft Engineers

Course: EGEN5208W Databases for Soft Engineers

Instructor: Abdelghny Orogat

Department: Department of Systems and Computer Engineering

University: Carleton University

Project Type: Individual Project (Solo)

Project Overview

Design and implement a **Health and Fitness Club Management System** using PostgreSQL as the backend database.

The system models the core operations of a small fitness club and supports exactly three user roles:

- **Members** – clients of the fitness club
- **Trainers** – staff who provide training services
- **Administrative Staff** – staff who manage memberships and operations

The focus of this project is database design, normalization, and SQL correctness, not advanced UI or full-stack complexity.

Important Clarifications

This is an **individual (solo) project**. Each student designs and implements their own database schema and application. The project must be self-contained and runnable by the instructor.

Project Scope

To avoid over- or under-engineering, the system scope is intentionally limited:

- The system manages memberships, trainers, sessions/classes, and administrative actions
- Payment gateways, mobile apps, external APIs, and real authentication systems are out of scope
- A simple login mechanism (username/password stored in the database) is sufficient

Project Requirements Summary

Requirement	Exact Expectation
Entities in ER Model	Minimum 6 entities (must be meaningful, not artificial)
Relationships in ER Model	Minimum 5 relationships, including at least one many-to-many
Application Operations	Exactly 8 total operations
Role Coverage	All 3 roles must be represented
Operation Distribution	4 Member + 2 Trainer + 2 Admin
Database Features	At least 1 View, 1 Trigger, 1 Index (excluding PK indexes)
Database System	PostgreSQL (mandatory)
Normalization	Schema must be in 3NF or higher

Operations Constraint

To ensure fairness and comparable workload:

- You must implement **exactly 8 operations**
- Each operation must:
 - Perform at least one meaningful SQL action
 - Interact with real database tables
 - Be executable independently
- Adding extra operations will not increase marks.

Database Features

View: Must combine data from at least two tables and support a realistic query use case.

Trigger: Must enforce a business rule (e.g., availability, capacity, automatic status update).

Index: Must improve performance for a realistic query (not just a primary key).

Technology Flexibility

You may use any programming language or framework. You may use raw SQL or an ORM. The grading emphasis is on:

- Correct schema design
- SQL quality
- Constraint enforcement
- Data integrity

Deliverables Overview

Full details are provided in the linked documents below. At a minimum, students will submit:

- Database design artifacts
- SQL or ORM implementation
- Working application demonstrating all operations
- Documentation and demo evidence

Documentation Navigation

Each section below must be read carefully before implementation:

1. [Problem Statement](#) – Business context and system boundaries
2. [Functional Requirements](#) – Exact definition of the 8 required operations
3. [ER Model Requirements](#) – Design rules and constraints
4. [Database Schema & SQL Requirements](#) – Tables, keys, constraints
5. [Implementation Requirements](#) – Application and coding rules
6. [Submission Requirements](#) – What to submit and how
7. [Grading Criteria](#) – How marks are allocated